Name: Shravani Kurkute **PRN:** 22311926
Roll No.: 421129 **Div:** Btech-A
Subject: NLP

Assignment No. 3

Code:

```
import random
import json
from collections import defaultdict, Counter

import nltk
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from nltk.corpus import treebank
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
                             classification_report, confusion_matrix)

RANDOM_SEED = 42
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

for package_name in ["treebank", "universal_tagset", "punkt", "punkt_tab"]:
    nltk.download(package_name, quiet=True)

# Load and split data
from nltk.tag.mapping import map_tag

tagged_sentences = [[(w, map_tag("en-ptb", "universal", t)) for w, t in sent if t != "-NONE-"]
        for sent in treebank.tagged_sents()]
tagged_sentences = [s for s in tagged_sentences if s]
random.shuffle(tagged_sentences)
split_index = int(0.8 * len(tagged_sentences))
training_sentences, testing_sentences = tagged_sentences[:split_index],
tagged_sentences[split_index:]
print(f"Total sentences: {len(tagged_sentences)} | Train: {len(training_sentences)} | Test:
{len(testing_sentences)}")
```

```
# HMM tagger
class HiddenMarkovTagger:
    START, END = "<s>", "</s>"

    def __init__(self, smoothing_factor=1e-3, max_suffix_length=3):
        self.smoothing_factor = smoothing_factor
        self.max_suffix_length = max_suffix_length

    def train_model(self, tagged_sentence_data):
        transition_counts = defaultdict(Counter)
        emission_counts = defaultdict(Counter)
        suffix_counts = defaultdict(Counter)
        tag_counts = Counter()
        word_counts = Counter(w.lower() for s in tagged_sentence_data for w, _ in s)

        for sent in
            tagged_sentence_data:
            previous_tag = self.START
            for w, t in sent:
                normalized_word = w.lower()
                transition_counts[previous_tag][t] += 1
                emission_counts[t][normalized_word] += 1
                tag_counts[t] += 1
                if word_counts[normalized_word] <= 2:
                    for suffix_length in range(1, self.max_suffix_length + 1):
                        suffix_counts[t][normalized_word[-suffix_length:]] += 1
                previous_tag = t
            transition_counts[previous_tag][self.END] += 1

        self.tags = sorted(tag_counts)
        self.vocab = set(word_counts)
        self.tag_counts = tag_counts
        number_of_tags = len(self.tags)

        self.log_transition_probs = {}
        for p in [self.START] + self.tags:
            total = sum(transition_counts[p].values()) + self.smoothing_factor * (number_of_tags +
1)
            self.log_transition_probs[p] = {t: np.log((transition_counts[p][t] + self.smoothing_factor)
/ total)
```

```
                    for t in self.tags + [self.END]}

        self.log_emission_probs = {t: {w: np.log(c / tag_counts[t]) for w, c in
emission_counts[t].items()}
                    for t in self.tags}

        self.suffix_tag_counts = suffix_counts
        self.suffix_totals = Counter()
        for t in self.tags:
            for s, c in suffix_counts[t].items():
                self.suffix_totals[s] += c
        total_tag_count = sum(tag_counts.values())
        self.tag_priors = {t: tag_counts[t] / total_tag_count for t in self.tags}
        return self

    def _emission_score(self, tag, word):
        normalized_word = word.lower()
        if normalized_word in self.vocab:
            return self.log_emission_probs[tag].get(normalized_word, np.log(1e-8))
        if any(ch.isdigit() for ch in normalized_word):
            return np.log(0.9) if tag == "NUM" else np.log(1e-6)
        for suffix_length in range(self.max_suffix_length, 0, -1):
            s = normalized_word[-suffix_length:]
            if self.suffix_totals[s] > 0:
                p = (self.suffix_tag_counts[tag][s] + 0.1) / (self.suffix_totals[s] + 0.1 * len(self.tags))
                bonus = 0.0
                if word[:1].isupper():
                    bonus = np.log(5.0) if tag == "NOUN" else 0.0
                return np.log(p) + bonus
        return np.log(self.tag_priors[tag])

    def decode_tags(self, words):
        sequence_length, number_of_tags_for_decode = len(words), len(self.tags)
        viterbi_scores = np.full((sequence_length, number_of_tags_for_decode), -np.inf)
        backpointers = np.zeros((sequence_length, number_of_tags_for_decode), dtype=int)

        for tag_index, t in enumerate(self.tags):
            viterbi_scores[0, tag_index] = self.log_transition_probs[self.START][t] +
self._emission_score(t, words[0])
```

```
        for token_index in range(1, sequence_length):
            for tag_index, t in enumerate(self.tags):
                emission_value = self._emission_score(t, words[token_index])
                candidate_scores = [viterbi_scores[token_index - 1, previous_tag_index] +
self.log_transition_probs[self.tags[previous_tag_index]][t] for previous_tag_index in
range(number_of_tags_for_decode)]
                backpointers[token_index, tag_index] = int(np.argmax(candidate_scores))
                viterbi_scores[token_index, tag_index] = max(candidate_scores) + emission_value

        final_scores = [viterbi_scores[sequence_length - 1, previous_tag_index] +
self.log_transition_probs[self.tags[previous_tag_index]][self.END] for previous_tag_index in
range(number_of_tags_for_decode)]
        best_tag_index = int(np.argmax(final_scores))
        best_path = [best_tag_index]
        for token_index in range(sequence_length - 1, 0, -1):
            best_tag_index = backpointers[token_index, best_tag_index]
            best_path.append(best_tag_index)
        return [self.tags[tag_index] for tag_index in reversed(best_path)]

    def tag_words(self, sentence):
        words = nltk.word_tokenize(sentence) if isinstance(sentence, str) else sentence
        return list(zip(words, self.decode_tags(words)))

# Baseline tagger
def make_baseline_tagger(training_data):
    word_tag_counts = defaultdict(Counter)
    for s in training_data:
        for w, t in s:
            word_tag_counts[w.lower()][t] += 1
    fallback_tag = Counter(t for s in training_data for _, t in s).most_common(1)[0][0]
    return lambda words: [word_tag_counts[w.lower()].most_common(1)[0][0] if w.lower() in
word_tag_counts else fallback_tag
                          for w in words]

# Train and evaluate
hmm_tagger = HiddenMarkovTagger(smoothing_factor=1e-3,
max_suffix_length=3).train_model(training_sentences)
baseline_tagger = make_baseline_tagger(training_sentences)

true_tags, predicted_tags, baseline_predictions = [], [], []
```

```
tagging_errors = []
unseen_token_count = correctly_tagged_unseen_tokens = 0
for sent in testing_sentences:
    words = [w for w, _ in sent]
    gold_tags = [t for _, t in sent]
    predicted_tag_sequence = hmm_tagger.decode_tags(words)
    baseline_tag_sequence = baseline_tagger(words)
    true_tags += gold_tags
    predicted_tags += predicted_tag_sequence
    baseline_predictions += baseline_tag_sequence
    for token_index, (w, g, p) in enumerate(zip(words, gold_tags, predicted_tag_sequence)):
        if w.lower() not in hmm_tagger.vocab:
            unseen_token_count += 1
            correctly_tagged_unseen_tokens += (g == p)
        if g != p:
            context_window = " ".join(words[max(0, token_index - 3): token_index + 4])
            tagging_errors.append((w, p, g, context_window))

hmm_accuracy = accuracy_score(true_tags, predicted_tags)
baseline_accuracy = accuracy_score(true_tags, baseline_predictions)
macro_precision, macro_recall, macro_f1, _ = precision_recall_fscore_support(true_tags,
predicted_tags, average="macro", zero_division=0)
weighted_precision, weighted_recall, weighted_f1, _ =
precision_recall_fscore_support(true_tags, predicted_tags, average="weighted",
zero_division=0)

print("\n================ EVALUATION ================")
print(f"Test tokens            : {len(true_tags)}")
print(f"Baseline (most-freq) acc: {baseline_accuracy:.4f}")
print(f"HMM + Viterbi accuracy : {hmm_accuracy:.4f}")
print(f"OOV tokens             : {unseen_token_count} ({unseen_token_count
/ len(true_tags):.2%}), "
     f"OOV accuracy = {correctly_tagged_unseen_tokens / max(unseen_token_count,
1):.4f}") print(f"Macro         P/R/F1  : {macro_precision:.4f} / {macro_recall:.4f} /
{macro_f1:.4f}") print(f"Weighted P/R/F1               : {weighted_precision:.4f} /
{weighted_recall:.4f} /
{weighted_f1:.4f}")
print("\nPer-tag report:\n")
print(classification_report(true_tags, predicted_tags, digits=4, zero_division=0))

# Sample predictions
```

```
sample_sentences = [
    "The quick brown fox jumps over the lazy dog .",
    "She sells sea shells by the sea shore .",
    "Pune is a beautiful city in Maharashtra
    .", "I will book a flight to Delhi tomorrow
    .", "Please read the book that she wrote .",
    "The company reported strong profits in 2023 .",
    "Students are learning natural language processing quickly .",
    "The government announced new economic policies yesterday .",
    "He runs five miles every morning before breakfast .",
    "The stock market fell sharply after the report .",
    "Mary gave her brother a gift for his birthday .",
    "Scientists have discovered a new species of frog .",
    "The committee will meet on Friday to discuss the budget .",
    "They are building a modern hospital near the river .",
    "My sister works for a large software company in Bangalore .",
    "The weather was cold and the roads were slippery .",
    "Investors expect interest rates to rise next quarter .",
    "Can you help me find the nearest railway station ?",
    "The teacher asked the students to submit their assignments .",
    "Several banks have reduced their lending rates .",
    "The old man walked slowly across the crowded street .",
    "Microsoft and Google are investing heavily in artificial intelligence .",
    "We can not ignore the effects of climate change .",
    "The new law will take effect in January .",
]

print("\n================ SAMPLE OUTPUT ================")
sample_outputs = []
for s in sample_sentences:
    tagged_words = hmm_tagger.tag_words(s.split())
    formatted_output = " ".join(f"{w}/{t}" for w, t in tagged_words)
    sample_outputs.append((s, formatted_output))
    print(f"\nInput : {s}\nOutput: {formatted_output}")

# Error analysis
confusion_pair_counts = Counter((p, g) for _, p, g, _ in tagging_errors)
print("\n================ TOP CONFUSIONS (pred -> gold) ================")
for (p, g), c in confusion_pair_counts.most_common(8):
    print(f"{p:>5} -> {g:<5} : {c}")
```

```
print("\n================ SAMPLE ERRORS ================")
for w, p, g, context_window in tagging_errors[:15]:
    print(f"{w:<15} pred={p:<5} gold={g:<5} | ...{context_window}...")

# Evaluation plots
tag_labels = hmm_tagger.tags
confusion_matrix_counts = confusion_matrix(true_tags, predicted_tags, labels=tag_labels)
normalized_confusion_matrix = confusion_matrix_counts /
confusion_matrix_counts.sum(axis=1, keepdims=True).clip(min=1)
fig, ax = plt.subplots(figsize=(8, 6.5))
im = ax.imshow(normalized_confusion_matrix, cmap="Blues")
ax.set_xticks(range(len(tag_labels)))
ax.set_yticks(range(len(tag_labels)))
ax.set_xticklabels(tag_labels, rotation=45, ha="right")
ax.set_yticklabels(tag_labels)
for token_index in range(len(tag_labels)):
    for tag_index in range(len(tag_labels)):
        if confusion_matrix_counts[token_index, tag_index]:
            ax.text(tag_index, token_index, confusion_matrix_counts[token_index, tag_index],
ha="center", va="center", fontsize=7,
                    color="white" if normalized_confusion_matrix[token_index, tag_index] > 0.5 else
"black")
ax.set_xlabel("Predicted tag")
ax.set_ylabel("Gold tag")
ax.set_title("Confusion Matrix - HMM + Viterbi (Universal tagset)")
fig.colorbar(im, fraction=0.046)
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.close()

per_tag_precision, per_tag_recall, per_tag_f1, _ = precision_recall_fscore_support(true_tags,
predicted_tags, labels=tag_labels, zero_division=0)
x = np.arange(len(tag_labels))
plt.figure(figsize=(9, 4.5))
plt.bar(x - 0.27, per_tag_precision, 0.27, label="Precision")
plt.bar(x, per_tag_recall, 0.27, label="Recall")
plt.bar(x + 0.27, per_tag_f1, 0.27, label="F1")
plt.xticks(x, tag_labels, rotation=45)
plt.ylim(0, 1.05)
```

```
plt.title("Per-tag Precision / Recall / F1")
plt.legend()
plt.tight_layout()
plt.savefig("per_tag_scores.png", dpi=150)
plt.close()

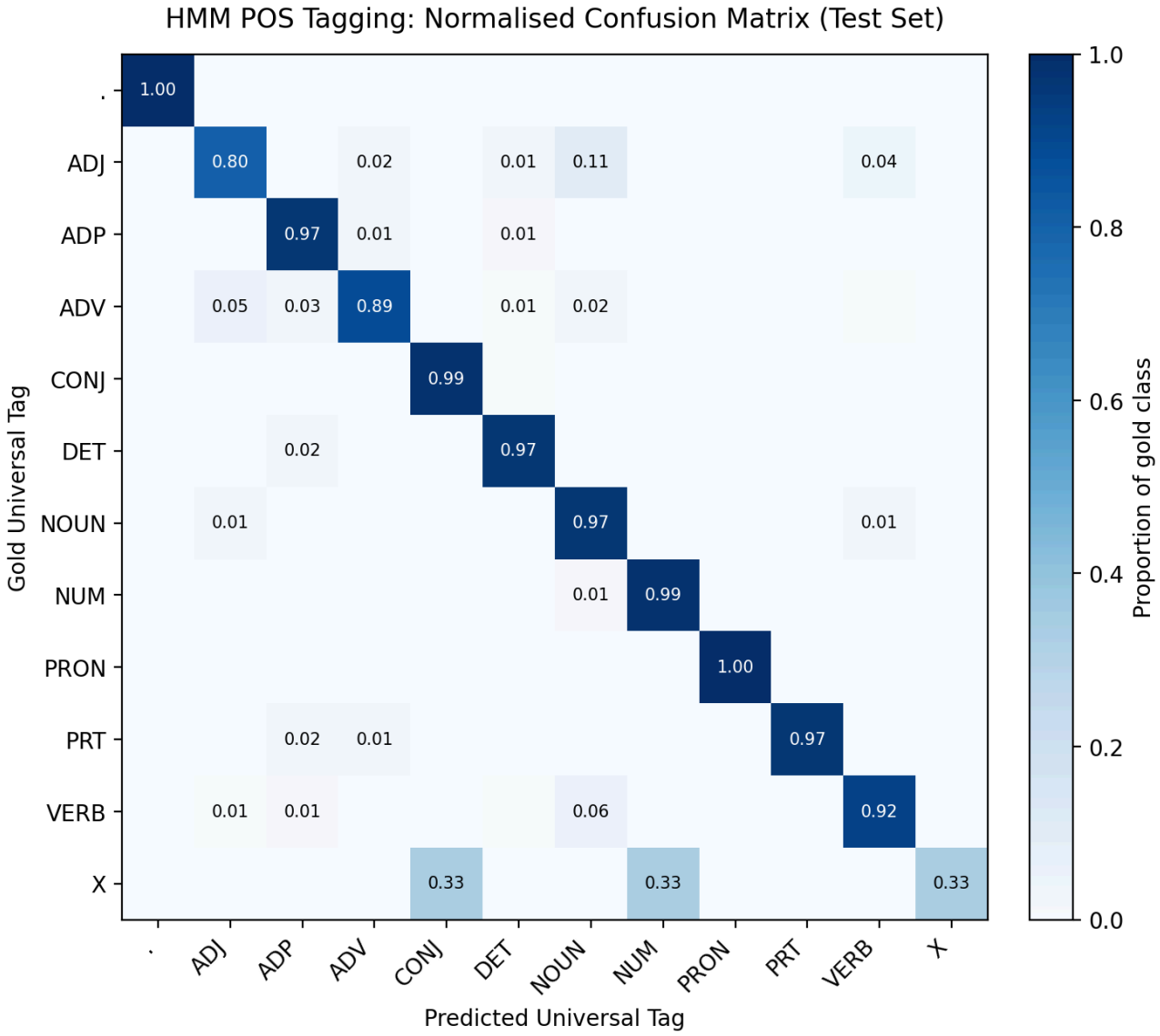
# Save evaluation results
json.dump({
    "total": len(tagged_sentences), "train": len(training_sentences), "test": len(testing_sentences),
    "test_tokens": len(true_tags), "base_acc": baseline_accuracy, "acc": hmm_accuracy,
    "oov_total": unseen_token_count, "oov_acc": correctly_tagged_unseen_tokens /
max(unseen_token_count, 1),
    "macro": [macro_precision, macro_recall, macro_f1], "weighted": [weighted_precision,
weighted_recall, weighted_f1],
    "samples": sample_outputs,
    "errors": tagging_errors[:40],
    "top_confusions": [[p, g, c] for (p, g), c in confusion_pair_counts.most_common(8)],
    "tags": tag_labels,
}, open("results.json", "w"), indent=1)
print("\nSaved: confusion_matrix.png, per_tag_scores.png, results.json")

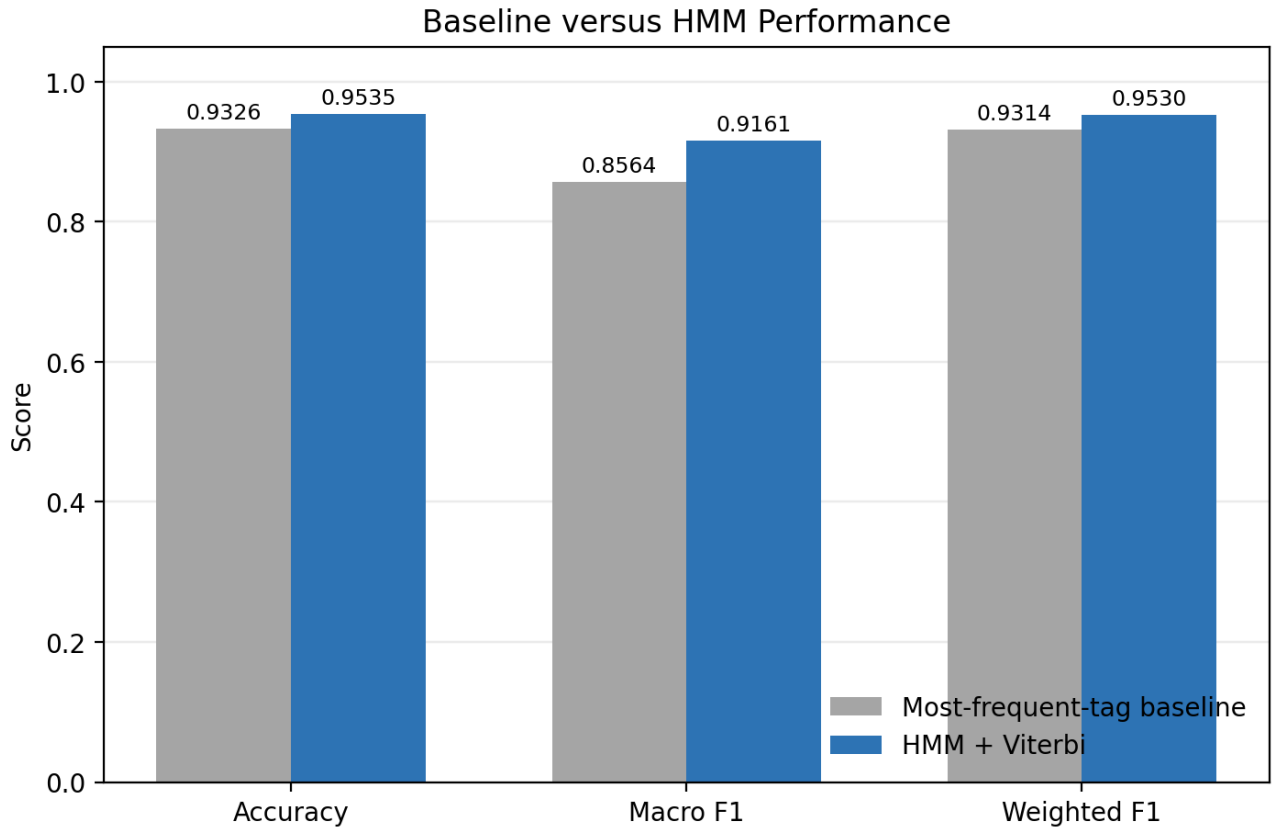
# Save train and test data
with open("train_sentences.txt", "w", encoding="utf-8") as f:
    for s in training_sentences:
        f.write(" ".join(f"{w}/{t}" for w, t in s) + "\n")

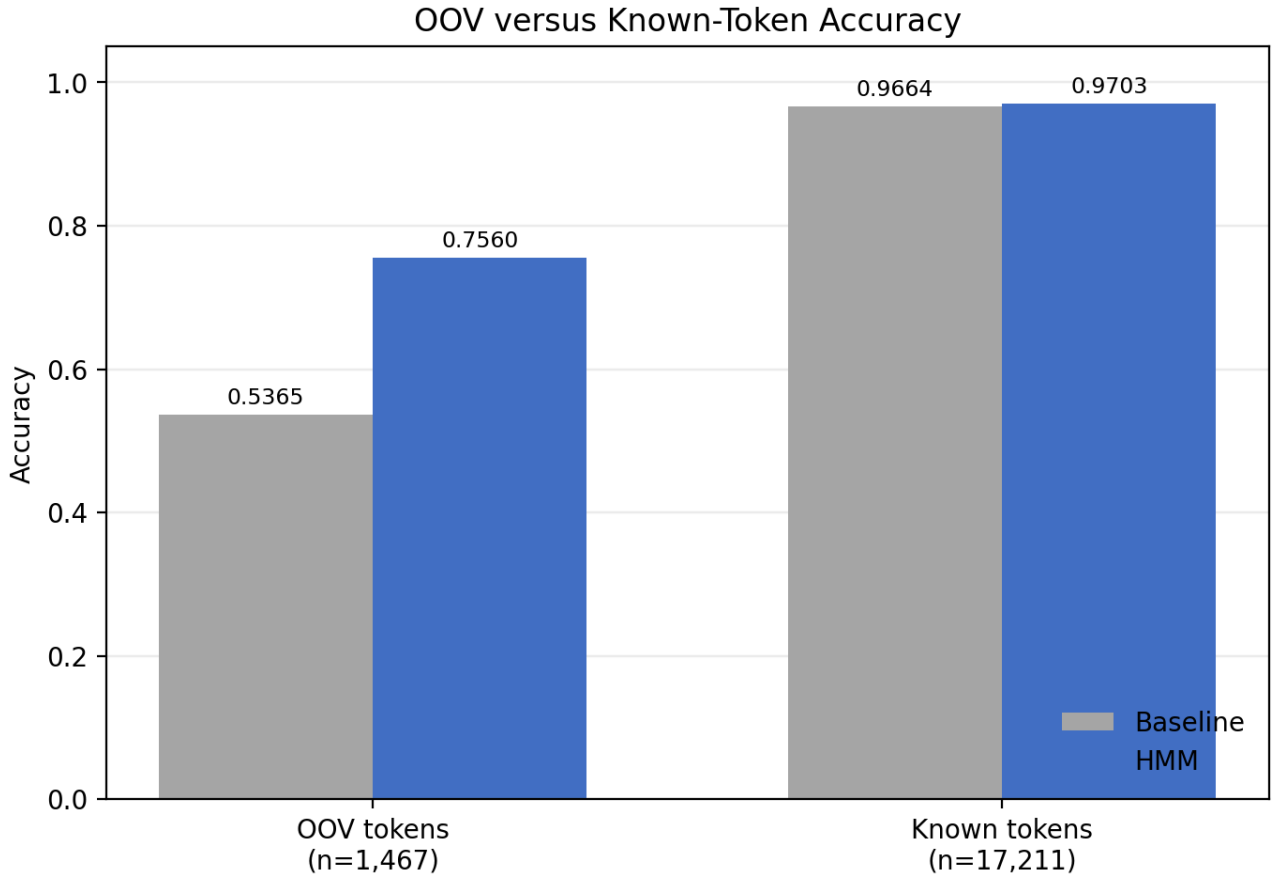
with open("test_sentences.txt", "w", encoding="utf-8") as f:
    for s in testing_sentences:
        f.write(" ".join(f"{w}/{t}" for w, t in s) + "\n")

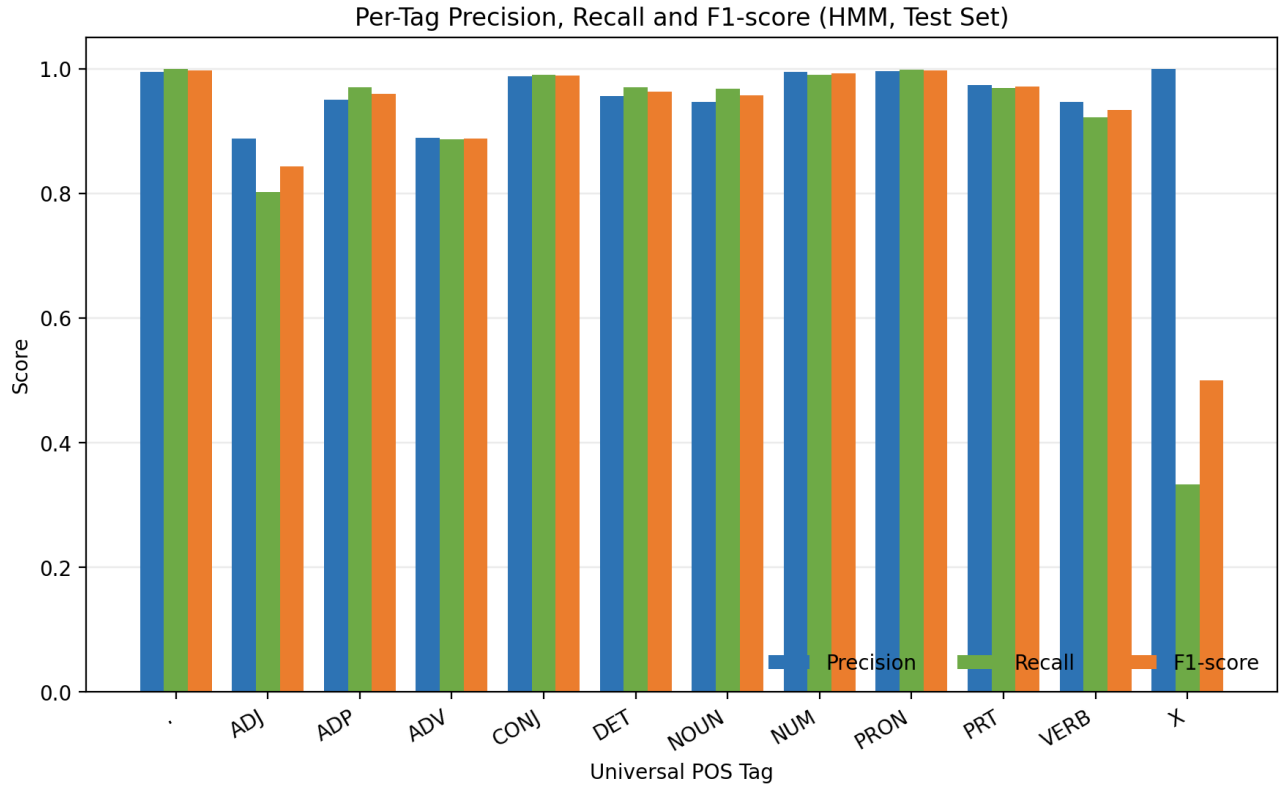
with open("custom_25_sentences.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(sample_sentences))
```

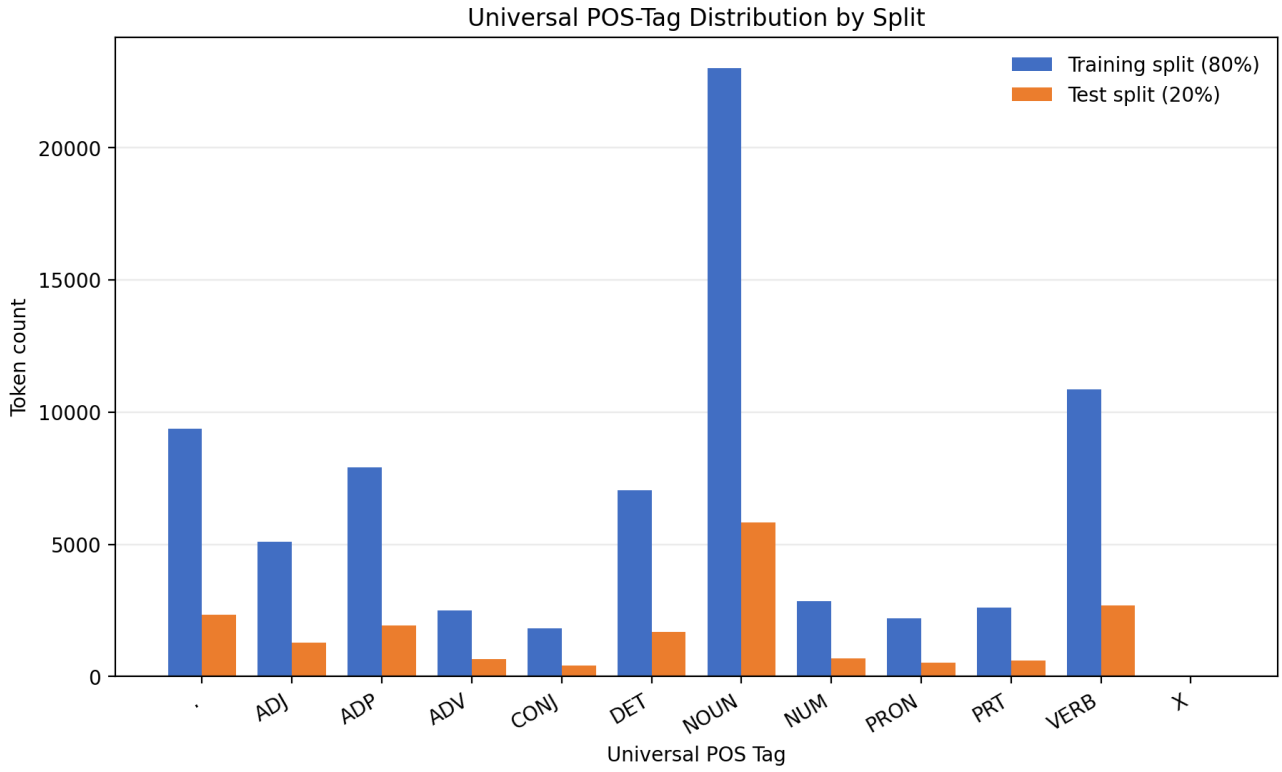
Output:


HMM POS Tagging: Normalised Confusion Matrix (Test Set)
Gold Universal Tag
Predicted Universal Tag
Proportion of gold class
.
ADJ
ADP
ADV
CONJ
DET
NOUN
NUM
PRON
PRT
VERB
X
1.00
0.80 0.02 0.01 0.11 0.04
0.97 0.01 0.01
0.05 0.03 0.89 0.01 0.02
0.99
0.02 0.97
0.01 0.97 0.01
0.01 0.99
1.00
0.02 0.01 0.97
0.01 0.01 0.06 0.92
0.33 0.33 0.33
0.0
0.2
0.4
0.6
0.8
1.0


Baseline versus HMM Performance
Score
1.0
0.8
0.6
0.4
0.2
0.0
0.9326
0.9535
0.8564
0.9161
0.9314
0.9530
Most-frequent-tag baseline
HMM + Viterbi
Accuracy
Macro F1
Weighted F1


OOV versus Known-Token Accuracy
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
0.5365
0.7560
0.9664
0.9703
Baseline
HMM
OOV tokens
(n=1,467)
Known tokens
(n=17,211)


Per-Tag Precision, Recall and F1-score (HMM, Test Set)
Score
1.0
0.8
0.6
0.4
0.2
0.0
Precision
Recall
F1-score
.
ADJ
ADP
ADV
CONJ
DET
NOUN
NUM
PRON
PRT
VERB
X
Universal POS Tag


Universal POS-Tag Distribution by Split
Token count
20000
15000
10000
5000
0
Training split (80%)
Test split (20%)
.
ADJ
ADP
ADV
CONJ
DET
NOUN
NUM
PRON
PRT
VERB
X
Universal POS Tag